

TEAM WORKSHOP · PART 2

Docker Compose

The by-hand stack from Part 1, tamed: one committed file, one command.

prerequisite: the Docker basics workshop (images, docker run, volumes, networks)

On this call

1 · The stack as a file ~20 min

Why Compose exists and where it sits, what `up` really does, the `compose.yaml` walkthrough — and the whole stack running live.

2 · The building blocks ~25 min

Networks, volumes, healthchecks, the three `depends_on` conditions, build targets — the Compose half of each, with a migration job and a deliberate crash.

3 · Team patterns ~25 min

Config with `.env`, the daily-driver commands and their gotchas, the override file, profiles, Compose in CI.

4 · Debugging & ops ~15 min

The stack-level toolbox, the Compose failure bestiary, shutdown and limits — then break it and find it. Q&A.

SECTION 1

The stack as a file

From four fragile commands to one committed file.

Why Compose

The `docker run` for our little API is already unbearable:

```
docker network create demo
docker run -d --name db --network demo -e POSTGRES_USER=... -e POSTGRES_PASSWORD=... \
-v pgdata:/var/lib/postgresql/data postgres:16-alpine
docker run -d --name api --network demo -p 8000:8000 -e DATABASE_URL=... \
-v ./app:/code/app notes:v2.1 uvicorn ... --reload
```

- Compose puts the whole stack in one declarative file: `compose.yaml`
- One command up, one command down; networks created automatically
- The file is **committed** — the dev environment becomes code-reviewed and onboarding becomes `git clone && docker compose up`

Where Compose sits

Compose is

- one machine, one Docker daemon
- a **file** that describes services, and a CLI that makes the containers match it
- the right tool for **dev environments, CI, and a single VM** — most teams' real footprint

Compose is not

- a cluster: no scheduling across machines, no self-healing when a node dies
- rolling updates, autoscaling, load balancing between replicas
- that is Kubernetes (or Swarm) territory — same *ideas*, much more machinery

- A loose vocabulary bridge, **not** an equivalence: a Compose *service* is roughly what Kubernetes splits into a Deployment and a Service; a *healthcheck* is where the readiness-probe habit starts; a *named volume* is a first taste of storage outside the container; `.env` is client-side templating — Kubernetes moves that into ConfigMaps and Secrets
- What transfers is the thinking: services, health, config from outside, state outside the container. The file format does not

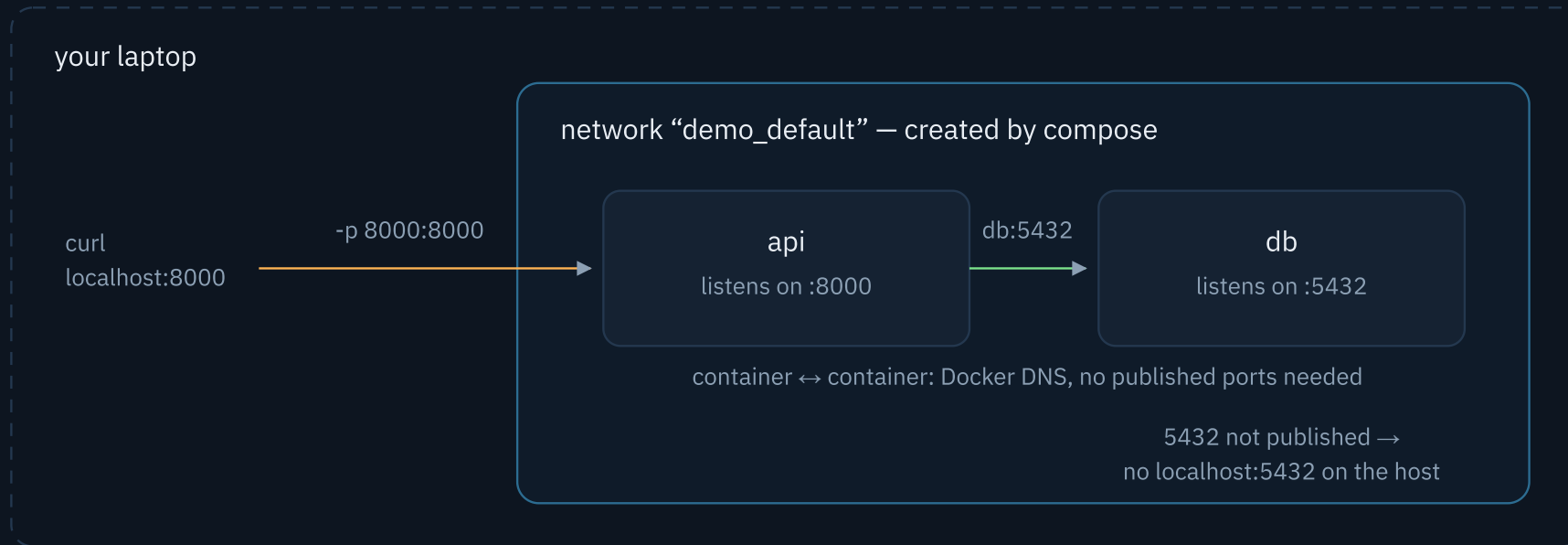
What `up` actually does

`docker compose up` is a **reconcile**, not a script: compare the file with what exists, fix the difference.

STEP	WHAT HAPPENS	YOU SEE
1	the project network and named volumes are created if missing	<code>Network demo_default Created</code>
2	images: build the services that have <code>build:</code> (cache permitting), pull the rest	<code>Image notes:compose Built</code>
3	containers are created — or recreated if their config changed since last time	<code>Container demo-api-1 Recreated</code>
4	started in dependency order, each gate honoured: started · healthy · completed	<code>db Healthy</code> → <code>migrate Exited</code> → <code>api Started</code>
5	foreground: attach to every log; <code>-d</code> : return; <code>--wait</code> : block until all gates are green	interleaved, prefixed logs

Unchanged services are left alone — that is why `up` is safe to run again and again, and why `restart` is rarely what you want.

The whole picture



Other compose projects get their own network — stacks are isolated from each other by default.

- Compose names everything after the **project**: containers `demo-api-1`, network `demo_default`, volume `demo_pgdata`. The name comes from the folder — or, as in our file, a top-level `name: demo` that pins it whatever the folder is called
- `-p other` and `COMPOSE_PROJECT_NAME` still win — that is how you run two copies side by side. Published ports are never isolated: two stacks on `8000:8000` still collide

compose.yaml, annotated

```
services:
  api:
    build: .                # build from our Dockerfile...
    image: notes:compose    # ...and call the result this
    ports: ["${API_PORT:-8000}:8000"] # publish to host (from .env, or 8000)
    environment:
      DATABASE_URL: postgresql://workshop:workshop@db:5432/notes
    depends_on: { migrate: { condition: service_completed_successfully } }
  migrate:
    image: notes:compose    # same image, different command, exits when done
    command: python -m app.migrate
    depends_on: { db: { condition: service_healthy } }
  db:
    image: postgres:16-alpine
    environment: { POSTGRES_USER: workshop, POSTGRES_PASSWORD: workshop, POSTGRES_DB: notes }
    volumes: [pgdata:/var/lib/postgresql/data] # named volume → state survives
volumes:
  pgdata:
```

- Three **services**, two images: `api` and `migrate` share one build (`build:` + `image:` = build it, name it), `db` is pulled. Compose is just `docker run` flags, versioned in a file
- Top-level `volumes:` *declares* the named volume; the line under `db` *mounts* it

Merged view, without dev extras or healthchecks — section 2 takes it apart, section 3 adds the override.

The whole stack, one command

```
$ docker compose up --build          # watch the order: db Healthy → migrate Exited → api Started
$ docker compose ps -a              # api, db: (healthy) – and migrate: Exited (0). -a, or you'd mis
$ curl -X POST "localhost:8000/notes?text=hello-team"
$ curl localhost:8000/notes
$ docker compose exec db psql -U workshop notes -c 'select * from notes'
$ docker compose exec api getent hosts db # DNS, seen live: db → 172.x.x.x
# edit app/main.py – uvicorn reloads through the bind mount. no rebuild.
```

Watch for: service names prefixing the logs · the gates in the startup order · reload on save.

SECTION 2

The building blocks

Networks, volumes, health, dependencies, builds — the half of each that belongs to Compose.

Networks: what Compose adds

Part 1: containers on one network reach each other by name. Compose decides **which networks exist** and **who is on them** — and that decides who can talk.

```
services:
  proxy:
    networks: [frontend]
    ports: ["80:80"]
  api:
    networks: [frontend, backend]
  db:
    networks: [backend]          # unreachable from anything that isn't on "backend"
networks:
  frontend:
  backend:
```

- No `networks:` anywhere → every service joins the project's `default` network. Fine for most stacks; split when something must **not** reach the database
- Isolation is **membership**, nothing else: `expose:` is documentation, `ports:` is a hole to the host
- Extra names: `networks: { backend: { aliases: [postgres] } }`. Reach your laptop from a container: `host.docker.internal` (Docker Desktop; on Linux add `extra_hosts: ["host.docker.internal:host-gateway"]`)
- `--scale api=2` with a fixed `"8000:8000"` collides on the host — publish `"8000"` alone for a random host port per replica; `docker compose port --index=1 api 8000` tells you which (repeat per index)

Volumes: what Compose adds

KIND	WHAT COMPOSE GIVES YOU	WATCH OUT
named <code>pgdata:</code>	declared at top level, created on <code>up</code> , prefixed <code>demo_pgdata</code> , survives <code>down</code>	<code>down -v</code> deletes it — the reset button and the footgun
bind <code>./app:/code/app</code>	relative to the compose file, not to your shell	on Linux the container writes as its UID — root-owned files in your repo
anonymous (<code>VOLUME</code> in an image, or <code>- /data</code>)	kept when <code>up</code> recreates the container — but nothing reattaches it after <code>down</code>	silent data loss on the next <code>up</code> : state you thought was kept, wasn't. Name it
<code>external: true</code>	reuse a volume Compose didn't create — shared between projects, or precious	<code>down -v</code> never touches it; you create and delete it yourself

`docker compose config` shows the resolved names, `docker volume ls` the reality. Remove volumes by name — never a blanket `volume prune` on a machine with other projects.

Healthcheck anatomy

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U workshop -d notes"]    # exit 0 = healthy. runs INSIDE the container
  interval: 2s        # how often
  timeout: 2s         # a hanging check counts as a failure
  retries: 15         # consecutive failures before "unhealthy"
  start_period: 10s   # grace: failures in here don't count, a success ends it early
```

```
create — (starting) — start_period —> healthy — retries × interval of failures —> unhealthy
      failures ignored                      still running! only ps, inspect, depends_on and --wait notice
```

- Whatever the check calls must exist in the image: `pg_isready` in postgres, `python` in ours — there is no `curl` in slim
- `docker compose ps` shows the state; `docker inspect -f '{{json .State.Health}}' demo-db-1` shows the last outputs — read them when a check never goes green
- Unhealthy does **not** restart anything by itself. It is a signal — for `depends_on`, `--wait`, `ps`, and for you

depends_on: three conditions

CONDITION	WAITS UNTIL	USE IT FOR
<code>service_started</code>	the container is running	a dependency with no warm-up — rare
<code>service_healthy</code>	its healthcheck passed	databases, brokers, anything that boots slower than it starts
<code>service_completed_successfully</code>	it exited with 0	one-shot jobs: schema migrations, seeders, asset builds

- Plain `depends_on: [db]` is the first row — Postgres the *process* starts before Postgres *accepts connections*. The API boots, connects, crashes. Works-on-a-fast-laptop flavor
- Our stack: `db` healthy → `migrate` runs and exits → `api` starts. Schema changes are a **job**, not app startup — and the job is idempotent, because `up` runs it every time
- The gates are for **startup only** — if db dies later, nothing tells the api. Reconnect in the app; a `restart:` policy only helps if the process exits
- Scripting: `up -d --wait` blocks until every gate is satisfied — or fails

build: inside Compose

```
# Dockerfile – named stages
FROM python:3.12-slim AS base
  ...deps (no dev group), code, CMD
FROM base AS dev
  RUN uv sync          # + pytest, httpx
  COPY tests/ tests/
FROM base AS runtime   # last = default
```

<pre># compose.yaml services: api: build: . image: notes:compose migrate: image: notes:compose</pre>	<pre># compose.override.yaml (dev) services: api: build: target: dev # compose.ci.yaml: the same 3</pre>
--	--

- One Dockerfile, several **targets** — the multi-stage trick from Part 1, now chosen per environment by whichever file loads: dev and CI get `dev` (pytest, the tests), a plain `-f compose.yaml` builds the lean `runtime`
- `build:` + `image:` names the result so other services can reuse it (`migrate`) — Compose builds before it creates any container
- `build.args:` for build-time knobs; `up --build` after Dockerfile changes, or `develop.watch` rules to rebuild when the lockfile changes
- The cache is still yours: the layer order from Part 1 decides whether `--build` takes one second or sixty

Jobs, networks, and a deliberate crash

```
$ docker compose ps -a                # migrate: Exited (0) – the job that ran before api
$ docker compose run --rm migrate      # run it again by hand: idempotent – "schema is up to date"
$ docker compose run --rm api pytest   # one-off container, same network, same env – the tests hit the real db
$ docker run --rm alpine getent hosts db # not on the project network: nothing
$ docker run --rm --network demo_default alpine getent hosts db # membership is the whole story

# break it: a slow database and no health gate (compose.race.yaml – opt-in only)
$ docker compose down -v
$ docker compose -f compose.yaml -f compose.override.yaml -f compose.race.yaml up
# api: "connection refused" → "Application startup failed". ps still says Up – until the healthcheck disagrees
```

Watch for: `run --rm` inheriting everything · the DNS lookup that finds nothing · a container that is Up and useless.

SECTION 3

Team patterns

Config, daily commands, and the files a team can actually live with.

Config: the .env file

```
# .env – NOT committed (.gitignore it; commit a .env.example instead)
POSTGRES_PASSWORD=s3cret
API_PORT=8001
# compose.yaml – committed, no values inside
services:
  api:
    ports: ["${API_PORT:-8000}:8000"]          # fallback when unset
    environment:
      DATABASE_URL: postgresql://workshop:${POSTGRES_PASSWORD:?set it in .env}@db:5432/notes  # refuse to start without it
```

- Compose reads `.env` next to the compose file **automatically** and fills `${VAR}` while loading, before merging. `${VAR:-default}` = fallback, `${VAR:?msg}` = fail loudly
- Precedence: your shell beats `.env`. Inside a service, `environment:` beats `env_file:`
- `.env` never enters a container by itself — `${VAR}` fills the *file*; `env_file: app.env` hands a whole file to the *container*. Different mechanisms
- Debugging: `docker compose config` prints the resolved file, `config --environment` the variables it used

TWO GOTCHAS

`.env` is local config, not a vault — values show in `config` and `docker inspect`; real secrets want Compose `secrets:`. And Postgres reads `POSTGRES_*` only into an **empty** volume: change the password and auth fails until `down -v`.

Compose, daily driver commands

COMMAND	WHAT
<code>docker compose up -d --build</code>	start in background; <code>--build</code> after Dockerfile changes, <code>--wait</code> to block until healthy
<code>docker compose ps -a</code>	stack status + health; <code>-a</code> includes exited containers (jobs!)
<code>docker compose logs -f api</code>	follow one service's logs
<code>docker compose exec api bash</code>	shell into a running service
<code>docker compose run --rm api ...</code>	one-off container on the stack's network: migrations, tests, scripts
<code>docker compose down</code>	stop & remove containers and networks (volumes survive)
<code>docker compose down -v</code>	...and delete the project's volumes — the “reset my database” move
<code>docker compose config</code>	print the final resolved file (env substituted, overrides merged)
<code>docker compose up --watch</code>	rebuild/sync per the <code>develop.watch</code> rules in the file, e.g. rebuild when <code>pyproject.toml</code> changes

Terminology: `docker compose` (the plugin — v2 onwards, v5 today) is the tool; the standalone `docker-compose` binary is v1, retired. `compose.yaml` is the preferred filename (`docker-compose.yml` still works) and the `version:` line at the top is obsolete — delete it.

Lifecycle gotchas

- `Ctrl+C` on a foreground `up` **stops** the containers, it doesn't remove them — `ps -a` shows them, `down` cleans up
- `up -d` again **recreates only what changed** in the file; `restart` reuses the old container — an edited environment: needs `up`, not `restart`
- Dockerfile changed? `up --build`. New image tag in the file? `pull`, then `up`
- `--wait` is the scripting form: `up -d --wait` returns when every gate is green — or fails
- `down` removes what **this project** created: its containers, its network, and with `-v` its declared volumes. Never someone else's

RULE OF THUMB

Not sure what state you're in? `docker compose ps -a`, then `docker compose config`. One shows what *is* running, the other what *would*.

The override file

`docker compose up` automatically merges `compose.yaml` with `compose.override.yaml` when one exists. Base file = the contract; override = dev conveniences:

```
# compose.yaml – committed, prod-shaped
services:
  api:
    build: .
    image: notes:compose
    ports: ["${API_PORT:-8000}:8000"]
    environment:
      DATABASE_URL: ...
    depends_on:
      migrate: { condition: service_completed_successfully }
```

```
# compose.override.yaml – dev-only extras
services:
  api:
    build: { target: dev }    # the stage with pytest
    volumes:
      - ./app:/code/app      # live reload...
    command: uvicorn app.main:app
      --reload --host 0.0.0.0
  adminer:                   # extra service,
    image: adminer            # opt-in via profile
    profiles: ["tools"]
    ports: ["127.0.0.1:8080:8080"]
```

- Merge rules: **maps merge, lists append, scalars replace** — `build:` gains a `target`, `volumes:` gains a mount, `command:` is replaced whole. Keyed lists (`ports`, `volumes`) merge on their target, so an override can *change* a mount, not just add one
- `profiles:` makes a service opt-in: `docker compose --profile tools up` — skipped otherwise (and `down` needs the profile too)
- Explicit stacks skip the magic: `docker compose -f compose.yaml -f compose.ci.yaml up` merges left → right, ignores the override file, and resolves relative paths from the **first** file. `config` shows the merged truth either way

Compose in CI

Same contract, no override, one three-line file for the CI-only difference:

```
$ docker compose -f compose.yaml -f compose.ci.yaml up -d --build --wait    # green, or fail fast
$ docker compose -f compose.yaml -f compose.ci.yaml run --rm api pytest      # one-off container, same network,
$ docker compose -f compose.yaml -f compose.ci.yaml logs api                # on failure: the evidence
$ docker compose -f compose.yaml -f compose.ci.yaml down -v                 # always – even when tests fail
```

- `compose.ci.yaml` says only `build: { target: dev }`. The base contract is untouched, the dev override never loads — explicit `-f` sees to that
- `--wait` turns healthchecks into the readiness signal, and the `migrate` gate means tests never race the schema
- `run --rm` gets the stack's network and env for free. When the command grows, a dedicated `tests` service is the next step
- `COMPOSE_FILE=compose.yaml:compose.ci.yaml` once, instead of repeating the flags

The repo runs exactly this: `demo/compose.ci.yaml`, `demo/tests/test_api.py`.

Override, .env, profiles, data

```
$ diff <(docker compose -f compose.yaml config) <(docker compose config) # exactly what the override adds
$ docker compose -p review config | grep 'name:' # project naming – nothing started
$ cp .env.example .env # a real .env – set API_PORT=8001 in it
$ docker compose config | grep published # "8001" – substituted from the file, no flags
$ docker compose up -d --wait # only api is Recreated: only its config changed
$ curl localhost:8001/notes; rm .env; docker compose up -d --wait # ...and back to 8000
$ docker compose --profile tools up -d --wait # adminer joins → localhost:8080, browse the notes table

# data outlives the containers:
$ docker compose --profile tools down; docker compose ps -a # gone (profile again, or adminer stays)
$ docker compose up -d --wait; curl localhost:8000/notes # still there – demo_pgdata survived
$ docker compose down -v; docker compose up -d --wait; curl localhost:8000/notes # [] – the reset button
```

Watch for: the reload you saw in demo 1 came from the override — the committed base has none of it •

Recreated only for the service whose config changed • **down -v** as the actual database reset.

SECTION 4

Debugging & ops

The Part 1 playbook, one level up: status → logs → config → exec, per stack.

Debugging a stack

COMMAND	QUESTION IT ANSWERS
<code>docker compose ps -a</code>	who's up, who's healthy, who exited — and with what code
<code>docker compose logs -f --since 10m api</code>	what it said — one service, or all of them interleaved
<code>docker compose config</code>	what Compose <i>thinks</i> the stack is: env substituted, overrides merged
<code>docker compose exec api sh</code>	look around inside a running service
<code>docker compose run --rm --no-deps api sh</code>	won't even start? same image and config, your command, no dependencies
<code>docker inspect -f '{{json .State.Health}}' demo-db-1</code>	why a healthcheck never goes green: its last outputs
<code>docker compose events</code>	the daemon's feed for this project: health_status, die, start
<code>docker compose top · stats</code>	processes and live CPU/memory, per service

Order: `ps -a` → `logs` → `config` → `exec` / `inspect`. Most Compose problems are in the file, not in the container.

Failure bestiary, Compose edition

dependency failed to start · didn't complete successfully

A gate stayed shut: a healthcheck never passed, or a job exited non-zero. `ps -a` for the culprit, `logs <that service>` for the reason. The dependent stays *Created* — on purpose.

port is already allocated

Another project publishes the same host port:

`docker ps --format '{{.Names}} {{.Ports}}'`. Fix: `API_PORT=...` in `.env`, or stop the other stack.

Found orphan containers for this project

You renamed or removed a service; its old container is still around.

`up --remove-orphans` or `down --remove-orphans`.

works after down -v, breaks after plain down

Stale state in a volume: a Postgres password changed in `.env`, a migration that assumed empty. Init variables only apply to an **empty** volume.

my change isn't there

Dockerfile changed but no `--build`; env changed but you ran `restart`; or you're in a folder with a different project. `config` shows what would run, `ps` what does.

network demo_default not found

Something still references a network that `down` removed — usually a stray `docker run --network` or a `run --rm` container that never got removed. `docker compose up` recreates it.

Shutdown, restarts, limits

Stopping cleanly

- `stop` and `down` send **SIGTERM**, wait `stop_grace_period` (10 s), then SIGKILL — the reason `down` sometimes takes ten seconds: your process ignored the signal
- PID 1 must handle signals (Part 1). Shell wrappers and some runtimes don't: `init: true` puts a tiny init in front
- `restart: unless-stopped` for things that should come back after a crash or a reboot. Dev stacks usually leave it off, so failures stay visible

Limits

```
services:
  api:
    cpus: 1.5
    mem_limit: 512m
    pids_limit: 200
```

- The same knobs as `docker run --memory --cpus`, now versioned. `docker compose stats` shows what is enforced
- `deploy.resources.limits` is the spec's spelling and Compose honours it too; the flat keys are the unambiguous ones on a laptop

Break it, then find it

```
# sabotage: a wrong password, the way it happens for real – a stray value in someone's shell
$ POSTGRES_PASSWORD=wrong docker compose up -d
#   service "migrate" didn't complete successfully: exit 1      ← the gate did its job
$ docker compose ps -a      # migrate Exited (1); api Created – never started, on purpose
$ docker compose logs migrate # "password authentication failed" – there it is
$ POSTGRES_PASSWORD=wrong docker compose config | grep DATABASE_URL # what it really ran with – same shell env
$ docker compose events --since 3m --until 1s | grep -E 'health_status|die' # the daemon's view, bounded
# fix = fix the config: plain up recreates what changed, and only that
$ docker compose up -d --wait && docker compose ps
```

Same playbook, one level up: **status** → **logs** → **config** → **exec/inspect**. Note that `db` was recreated too (its env changed) but the volume kept the old password — which is exactly why auth failed.

What you take home

1 · The stack as a file

`compose.yaml` is the dev environment — committed, reviewed, reproducible. `up` reconciles: it creates, recreates, and starts in gate order. Compose is one machine; the *ideas* scale to Kubernetes, the file doesn't.

2 · The building blocks

Isolation is network **membership**. State lives in **named** volumes. A healthcheck is a signal, not a restart. Three gates: started, **healthy**, **completed** — migrations are a job. One Dockerfile, targets per environment.

3 · Team patterns

Local config in `.env`, dev extras in `compose.override.yaml`, optional tools behind `profiles:`, CI on the bare contract with `--wait`. `up` recreates, `restart` doesn't; `down -v` resets.

4 · Debugging & ops

`ps -a` → `logs` → `config` → `exec`. Gates make failures loud and early. Volumes remember what containers forget.

IF YOU REMEMBER ONE THING

Onboarding should be `git clone && docker compose up` — everything else about the dev environment belongs in reviewed files, not in shell history.

FIN

Questions?

Then go compose something.